Virtual Audio Router

技术深度解析

Version 2.1.0

Virtual Audio Router — 技术深度解析

> 本文档详细整理项目中使用的所有核心技术，包括**技术背景**、**工作原理**和**项目中的具体实施方案**。

目录

1. WDM/WaveRT 内核音频驱动
2. Named Shared Memory (命名共享内存 IPC)
3. Lock-Free Ring Buffer (无锁环形缓冲区)
4. VST3 插件框架 + JUCE
5. WASAPI 音频接口
6. UDP 数据报传输
7. WebSocket 实时双向通信
8. SHA-1 哈希算法

9. QR Code 二维码生成
10. 编译期品牌切换系统

---

1. WDM/WaveRT 内核音频驱动

技术背景

WDM (Windows Driver Model) 是 Windows 操作系统的驱动架构。音频驱动最初使用 **WaveCyclic** 和 **WavePci** 端口模型，但这两种模型需要在内核和用户空间之间频繁复制数据，导致延迟较高。

WaveRT (Wave Real-Time) 是 Windows Vista 引入的新一代音频端口模型，核心思想是**让用户态应用直接访问驱动的 DMA 缓冲区**，消除数据复制，从而实现极低延迟。Windows 10/11 的 WASAPI Exclusive 模式就建立在 WaveRT 之上。

工作原理

```
┌──────────────────────────────────────────────┐
│   User Mode (应用层)                          │
│   ┌──────────┐    ┌──────────┐               │
│   │  DAW     │    │  Windows │               │
│   │  (VST3)  │    │  Audio   │               │
│   └────┬─────┘    │  Service │               │
│        │          └────┬─────┘               │
│========│===============│=====================│
│   Kernel Mode (内核层)  │                     │
│   ┌────┴───────────────┴──────┐               │
│   │  WaveRT Miniport Driver   │               │
│   │  ┌──────────────────┐     │               │
│   │  │  WaveRT Buffer   │◄──── 用户态直接映射 │
│   │  │  (DMA-like buffer) │   │               │
│   │  └──────────────────┘     │               │
│   └───────────────────────────┘               │
└──────────────────────────────────────────────┘
```

- 驱动创建 **Section Object**（共享内存段），映射到用户态
- 音频引擎 (audiodg.exe) 直接在映射的缓冲区上读写 PCM 数据
- 驱动通过 **Position Register** 通告当前播放/采集位置
- 没有 memcpy，没有 IOCTL 传输——真正的零拷贝

项目实施

```
文件: ASMRTOP_Driver_V2/Source/Main/
├── adapter.cpp       → 驱动入口点 (DriverEntry)
```

```
├── minwavert.cpp     → WaveRT Miniport 实现
├── minwavertstream.cpp → 音频流控制 (Play/Capture)
├── mintopo.cpp       → 拓扑 Miniport (音量/静音节点)
└── AsmrtopKernelIPC.h → 内核侧 IPC 桥接
```

关键实现： - 驱动注册 4 对扬声器 + 4 对麦克风 = **8 个音频端点** - 每个端点使用独立的 WaveRT buffer - 通过 `CSharedMemoryBridgeKernel` 在内核侧创建 Named Section，用于与 VST 插件通信 - 使用 `MmMapViewInSystemSpace()` 将共享内存映射到内核地址空间 - 使用 `MmProbeAndLockPages()` 锁定物理页面防止交换

---

2. Named Shared Memory (命名共享内存 IPC)

技术背景

进程间通信 (IPC) 有很多方式：管道、Socket、消息队列等。但对于需要在**高频率**下传输**大量数据**（如 48000Hz × 2ch × 4bytes = 384KB/s）且**延迟极低**的场景，**共享内存**是唯一选择。

Windows 的 **Named Section Object** 允许不同进程（甚至内核和用户态）通过一个全局名称访问同一块物理内存。

工作原理

```
                    物理内存
                 ┌──────────────┐
                 │ IpcAudioBuffer│
                 │ (1MB)        │
                 └──────┬───────┘
          ┌─────────────┼──────────────┐
          │             │              │
   ┌──────┴──────┐  ┌───┴────────┐  ┌──┴─────────┐
   │ 内核驱动    │  │ VST 插件 A │  │ VST 插件 B │
   │ (MapView    │  │ (MapView   │  │ (MapView   │
   │  InSystem) │  │  OfFile)   │  │  OfFile)   │
   └─────────────┘  └────────────┘  └────────────┘

命名: Global\VirtualAudioWDM_PLAY_0
           │         │      │
           │         │      └── channelId
           │         └── direction (PLAY/REC/P2P)
           └── baseName (品牌相关)
```

项目实施

内核侧 (AsmrtopKernelIPC.h):

```
// 使用 ZwCreateSection 创建命名段（全局可见）
ZwCreateSection(&m_SectionHandle, SECTION_ALL_ACCESS,
               &objectAttributes, &sectionSize,
               PAGE_READWRITE, SEC_COMMIT, NULL);
```

```
// 映射到内核地址空间
MmMapViewInSystemSpace(m_SectionObject, &m_SharedMemory, &viewSize);
```

用户侧 (AsmrtopIPC.h):

```
// 创建或打开命名共享内存
CreateFileMappingA(INVALID_HANDLE_VALUE, &sa,
                   PAGE_READWRITE, 0, sizeof(IpcAudioBuffer),
                   mappingName.toRawUTF8());

// 映射到用户进程地址空间
MapViewOfFile(hMapFile, FILE_MAP_ALL_ACCESS, 0, 0, sizeof(IpcAudioBuffer));
```

安全设计： - `SECURITY_DESCRIPTOR` 设置 NULL DACL → 允许所有用户访问 - 先尝试 `Global\` 前缀（跨 Session），失败则 `Local\`（同 Session） - 内核创建时用 `OBJ_KERNEL_HANDLE` 保护句柄

---

3. Lock-Free Ring Buffer (无锁环形缓冲区)

技术背景

音频处理中，**生产者-消费者**模式无处不在：驱动写，插件读；插件写，驱动读。传统方法用互斥锁 (Mutex) 保护共享数据，但锁的开销和优先级反转问题在实时音频中不可接受。

无锁环形缓冲区利用 CPU 的原子操作指令（如 `LOCK XCHG`、`memory fence`），实现了**单生产者-单消费者**场景下完全无锁的数据传输。

工作原理

```
Ring Buffer (131072 = 2^17 slots)

writePos=100                        readPos=90
     |                                   |
     ▼                                   ▼
┌──────────────────────────────────────────┐
|....|XXXXXXXXXX|.........................|....|
|    | 10 samples available               |    |
└──────────────────────────────────────────┘

     ← available = writePos - readPos = 10 →

索引计算: index = pos & 131071  (等价于 pos % 131072, 但更快)
```

关键约束： - 缓冲区大小必须是 **2 的幂**（131072 = 2^17） - 只有一个写者修改 `writePos`，一个读者修改 `readPos` - 使用 `memory_order_release` 写入，`memory_order_acquire` 读取 - 即使 writePos 溢出 uint32 也能正确计算 available（无符号减法）

项目实施

```
struct IpcAudioBuffer {
    std::atomic<uint32_t> writePos;  // 生产者维护
    std::atomic<uint32_t> readPos;   // 消费者维护
    float ringL[131072];             // 左声道 (~512KB)
    float ringR[131072];             // 右声道 (~512KB)
};
// 总大小: ~1MB per channel pair
```

写入（生产者）：

```
uint32_t w = writePos.load(memory_order_relaxed);  // 只自己写，relaxed够了
for (int i = 0; i < numSamples; ++i) {
    ringL[(w + i) & 131071] = left[i];   // 位运算取模
    ringR[(w + i) & 131071] = right[i];
}
writePos.store(w + numSamples, memory_order_release);  // release保证数据对消费者可见
```

读取（消费者）：

```
uint32_t w = writePos.load(memory_order_acquire);  // acquire看到生产者的最新写入
uint32_t r = readPos.load(memory_order_relaxed);
int32_t available = (int32_t)(w - r);  // 无符号减法，溢出安全
```

---

4. VST3 插件框架 + JUCE

技术背景

VST (Virtual Studio Technology) 是 Steinberg 1996 年发明的音频插件标准，让第三方开发者为 DAW 编写效果器和乐器。**VST3** 是 2008 年发布的第三代标准，改进了事件处理、动态 I/O 和组件分离。

JUCE (Jules' Utility Class Extensions) 是一个跨平台 C++ 框架，提供了 GUI、音频、网络等全套功能，是 VST 插件开发的事实标准工具。

工作原理

```
VST3 插件架构:

┌─ AudioProcessor ──────────────────┐
│  prepareToPlay(sampleRate, size) │  ← DAW 初始化
│  processBlock(AudioBuffer)       │  ← 实时音频回调 (每ms级)
│  getParameterValue()             │  ← 参数读取
│  setParameterValue()             │  ← 参数写入
└──────────────────────────────────┘
              │
              │ setProcessor()
              ▼
┌─ AudioProcessorEditor ────────────┐
│  paint(Graphics)                 │  ← UI 渲染
│  resized()                       │  ← 布局
│  timerCallback()                 │  ← 周期性 UI 更新
└──────────────────────────────────┘
```

关键约束： - `processBlock()` 在**实时音频线程**调用，禁止：分配内存、加锁、IO、系统调用 - Editor (UI) 在**消息线程**调用 - Processor 和 Editor 之间通过 `std::atomic` 通信

项目实施

本项目 4 个插件共 8 个类：

插件	Processor	Editor
WDM2VST	W2V_PluginProcessor	W2V_PluginEditor
VST2WDM	V2W_PluginProcessor	V2W_PluginEditor

插件	Processor	Editor
Send	Send_PluginProcessor	Send_PluginEditor
Receive	Recv_PluginProcessor	Recv_PluginEditor

参数管理： 使用 JUCE 的 `AudioProcessorValueTreeState` 实现参数持久化和 UI 绑定。

5. WASAPI 音频接口

技术背景

WASAPI (Windows Audio Session API) 是 Windows Vista+ 的原生音频接口，取代了 DirectSound 和 MME。它提供两种模式：

- **Shared Mode**：多个应用共享音频设备，Windows 混音器混合所有输出
- **Exclusive Mode**：独占设备，绕过混音器，最低延迟

工作原理

```
Shared Mode 数据流:

App A ┐
App B ┼→ Windows Audio Engine (audiodg.exe) → Audio Driver → Speaker
App C ┘       ↑ 混音 + 重采样 + 音量

Exclusive Mode:
App ──────────────────────────→ Audio Driver → Speaker
                                    (直通)
```

项目实施

WDM2VST 和 VST2WDM 使用 JUCE 封装的 WASAPI 接口进行实际的音频设备输入/输出：

```
// WDM2VST: 使用 AudioDeviceManager 打开 WASAPI 设备
deviceManager.initialise(2, 0, nullptr, true);  // 2 input channels

// audioDeviceIOCallback 在 WASAPI 线程上运行
void audioDeviceIOCallback(const float* const* inputChannelData, ...) {
    // 将 WASAPI 数据写入内部环形缓冲区
    // processBlock 从缓冲区读取
}
```

延迟模式实现：

```
enum LatencyMode { Safe=0, Fast, Extreme, Insane };
// Safe:    bufferSize = 384  (~8ms @ 48kHz)
// Fast:    bufferSize = 192  (~4ms)
```

```
// Extreme: bufferSize = 96   (~2ms)
// Insane:  bufferSize = 48   (~1ms)
```

6. UDP 数据报传输

技术背景

UDP (User Datagram Protocol) 是无连接的传输层协议。与 TCP 不同，UDP 不保证送达、不保证顺序、没有拥塞控制。但正是这些"缺点"使它适合实时音频：

- **无建连开销** → 低延迟
- **无重传** → 不会因等待丢包重传而卡顿
- **支持广播/组播** → 一对多传输

工作原理

```
UDP 音频包结构 (本项目自定义):

┌──────────────────────────────────┐
│ UdpPacketHeader (12 bytes)       │
│ ┌──────┬────┬────┬────┬────┬────┐ │
│ │"ASMR"│chId│ ch │ SR │ seq│size│ │
│ │4B    │1B  │1B  │2B  │2B  │2B  │ │
│ └──────┴────┴────┴────┴────┴────┘ │
├──────────────────────────────────┤
│ Audio Data (interleaved float32) │
│ [L0][R0][L1][R1]...[Ln][Rn]      │
│ max 480 samples × 2ch × 4B = 3840B │
└──────────────────────────────────┘

每包总大小: 12 + 3840 = 3852 bytes (< MTU 1500 会分片)
```

项目实施

发送端 (P2PTransport.h:L127):

```
// 分片发送：每包最多 480 samples (10ms @ 48kHz)
const int maxSamplesPerPacket = 480;
while (offset < numSamples) {
    int chunk = jmin(maxSamplesPerPacket, numSamples - offset);
    // 构造包头 + 交错音频数据
    udpSocket->write(udpTarget, udpPort, packet.data(), packet.size());
}
```

接收端 (P2PTransport.h:L602):

```
// 独立线程循环接收
while (!threadShouldExit()) {
    int ready = udpSocket->waitUntilReady(true, 500);  // 500ms 超时
    int bytesRead = udpSocket->read(buf.data(), buf.size(), false, senderIP,
senderPort);
    // 验证 magic="ASMR" + channelId 匹配
    // 写入本地 UDP ring buffer
}
```

广播支持： 目标地址设为 `192.168.x.255` 即可广播到整个子网。

---

7. WebSocket 实时双向通信

技术背景

WebSocket 是 HTML5 引入的全双工通信协议 (RFC 6455)。它从一个 HTTP 请求开始，通过 **Upgrade** 机制切换到二进制帧协议。浏览器原生支持，使得手机无需安装 App 即可接收实时数据。

工作原理

```
WebSocket 握手流程:

Client (手机浏览器)          Server (VST 插件)
    │                              │
    │── GET / HTTP/1.1 ──────────►│
    │   Upgrade: websocket         │
    │   Sec-WebSocket-Key: xxx     │
    │                              │
    │◄── HTTP/1.1 101 ────────────│
    │    Upgrade: websocket        │
    │    Sec-WebSocket-Accept:     │
    │    BASE64(SHA1(key+GUID))    │
    │                              │
    │◄═══ Binary Frame ═══════════ │  (音频数据)
    │◄═══ Binary Frame ═══════════ │
    │     ...持续推送...            |
```

WebSocket 帧格式:

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
┌─┬─────────────┬─┬─────────────┐
│F│   opcode    │M│  Payload len│  Extended payload length
│I│  (4 bits)   │A│  (7 bits)   │  (if len == 126: 2 bytes)
│N│             │S│             │  (if len == 127: 8 bytes)
│ │             │K│             │
├─┴─────────────┴─┴─────────────┤
│  Payload Data (audio PCM)      │
└────────────────────────────────┘

opcode: 0x81 = text frame, 0x82 = binary frame
```

项目实施

手工实现的 HTTP/WebSocket 服务器（无任何外部依赖）：

1. **TCP 监听** → `StreamingSocket::createListener(port)`
2. **HTTP 请求解析** → 判断 GET 请求 vs WebSocket Upgrade
3. **WebSocket 握手** → SHA-1(key + GUID) → Base64 编码
4. **二进制推送** → float32 → int16 → WebSocket binary frame

音频编码优化：

```
// float32 → int16 节省 50% 带宽
for (int i = 0; i < numSamples; ++i) {
    pcm16[i * 2]     = (int16_t)(left[i]  * 32767.0f);
    pcm16[i * 2 + 1] = (int16_t)(right[i] * 32767.0f);
}
```

嵌入式网页播放器： 服务器内嵌完整 HTML/CSS/JS 页面，包含： - AudioWorklet/ScriptProcessor 实时播放 - 波形可视化 (Canvas) - 电平表 (Peak Meter) - 音量控制 + 静音

---

8. SHA-1 哈希算法

技术背景

SHA-1 (Secure Hash Algorithm 1) 是 NIST 在 1995 年发布的密码学哈希函数，输入任意长度数据，输出 160 位 (20 字节) 摘要。WebSocket 协议规定必须使用 SHA-1 进行握手验证。

工作原理

```
输入 → [填充到 512bit 的倍数] → [分块处理] → 160-bit Hash

每个 512-bit 块的处理 (80 轮):
┌──────────────────────────────────┐
│  初始化 h0~h4 (5个32位寄存器)       │
│                                  │
│  for i = 0..79:                  │
│    if i < 20: f = (b&c)|((~b)&d)   │ ← Ch 函数
│    if 20≤i<40: f = b^c^d           │ ← Parity 函数
│    if 40≤i<60: f = (b&c)|(b&d)|(c&d)│ ← Maj 函数
│    if 60≤i<80: f = b^c^d           │ ← Parity 函数
│                                  │
│    temp = ROTL5(a) + f + e + K + W[i]│
│    e=d; d=c; c=ROTL30(b); b=a; a=temp│
│                                  │
│  h0+=a, h1+=b, h2+=c, h3+=d, h4+=e │
└──────────────────────────────────┘
```

项目实施

在 [P2PTransport.h:L370](#) 中**完整内联实现了 SHA-1**，仅用于 WebSocket 握手：

```
static void computeSHA1(const uint8_t* data, int len, uint8_t out[20]) {
    uint32_t h0=0x67452301, h1=0xEFCDAB89, h2=0x98BADCFE,
             h3=0x10325476, h4=0xC3D2E1F0;
    // ... 填充 + 80轮压缩 ...
}
```

用途： `Accept = Base64(SHA1(ClientKey + "258EAFA5-E914-47DA-95CA-C5AB0DC85B11"))`

9. QR Code 二维码生成

技术背景

QR Code (Quick Response Code) 是 1994 年由日本 Denso Wave 发明的二维码标准 (ISO/IEC 18004)。它使用 **Reed-Solomon 纠错码**在 **GF(256) 有限域**上运算，即使部分损坏也能恢复数据。

工作原理

```
QR Code 生成流程:

URL字符串
    │
    ▼
┌─────────┐     ┌─────────┐     ┌─────────┐
│ 数据编码 │ ──► │ 纠错编码 │ ──► │ 矩阵排列 │
│ (Byte模式)│     │ (Reed-  │     │ + 掩码  │
│         │     │ Solomon) │     │         │
└─────────┘     └─────────┘     └─────────┘
                     │
                     ▼
              GF(256) 有限域运算
              多项式除法 → ECC字节
```

GF(256) 有限域： 在模不可约多项式 `x^8 + x^4 + x^3 + x + 1` (0x11D) 下的有限域，每个元素 0~255，加法=XOR，乘法=查表/循环。这个域的特殊性质使得 Reed-Solomon 编码成为可能。

项目实施

在 QRCode.h 中**从零实现了完整的 QR 码生成器**（~316 行，无外部依赖）：

```
// GF(256) 乘法
static uint8_t gfMul(uint8_t x, uint8_t y) {
    int z = 0;
    for (int i = 7; i >= 0; i--) {
        z = (z << 1) ^ ((z >> 7) * 0x11D);  // 模不可约多项式
        z ^= ((y >> i) & 1) * x;
    }
    return (uint8_t)z;
}
```

```
// Reed-Solomon 编码
static std::vector<uint8_t> rsEncode(const std::vector<uint8_t>& data, int numEcc) {
    // 构造生成多项式 g(x) = (x-α^0)(x-α^1)...(x-α^(numEcc-1))
    // 多项式长除法得到余式 = ECC 字节
}
```

支持版本 1~6，纠错等级 L。 版本自动选择以容纳 URL 长度。

---

10. 编译期品牌切换系统

技术背景

同一个代码库需要输出两个品牌的产品（品牌版 + 公版），但不想维护两套代码。C/C++ 的**预处理器条件编译**（`#ifdef`）可以在编译时切换代码路径。

工作原理

```
PluginBranding.h:

                    ┌──────────────┐
 set_brand.ps1 ──► │ #define      │ ──► CMake 读取 ──► 编译
 (asmrtop/public)   │ ASMRTOP_BRAND│      ↓ 设置
                    └──────────────┘      PRODUCT_NAME
                                          COMPANY_NAME
```

项目实施

三层品牌控制：

层	文件	控制内容
C 宏	`PluginBranding.h`	运行时字符串（IPC名称、UI标题）
CMake	`CMakeLists.txt`	编译时元数据（VST3名称、开发商）
INX	`VirtualAudioDriver.inx`	驱动安装信息（设备名、声道名）

`set_brand.ps1` 一键切换所有三层，使用正则替换文件内容。

技术栈总览图

```
┌──────────────────────────────────────────────────────┐
│                    应用层                            │
│  ┌──────────────────────────────────────────────┐   │
│  │  JUCE C++ Framework                          │   │
│  │  ├── VST3 AudioProcessor / Editor            │   │
│  │  ├── WASAPI AudioDeviceManager               │   │
│  │  ├── StreamingSocket (TCP)                    │   │
│  │  ├── DatagramSocket (UDP)                     │   │
│  │  ├── Graphics / Component (GUI)               │   │
│  │  └── Thread / CriticalSection                 │   │
│  └──────────────────────────────────────────────┘   │
│                                                      │
│  ┌──────────────┐ ┌────────┐ ┌──────────────────┐  │
│  │ P2PTransport │ │QRCode  │ │ Localization     │  │
│  │ ├─ IPC       │ │(GF256) │ │ (UTF-8 + i18n)   │  │
│  │ ├─ UDP       │ │(R-S)   │ │                  │  │
│  │ └─ WebSocket │ │        │ │                  │  │
│  │   ├─ HTTP    │ └────────┘ └──────────────────┘  │
│  │   ├─ SHA-1   │                                   │
│  │   └─ AudioWorklet (JS)                           │
│  └──────────────┘                                   │
├═══════════════════════════════════════════════════════┤
│                    内核层                            │
│  ┌──────────────────────────────────────────────┐   │
│  │  WaveRT Miniport Driver                      │   │
│  │  ├── Named Section (Shared Memory)            │   │
│  │  ├── Lock-Free Ring Buffer                    │   │
│  │  ├── PCM Format Conversion (16/24/32bit)      │   │
│  │  └── MmMapViewInSystemSpace                   │   │
│  └──────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────┘
```